

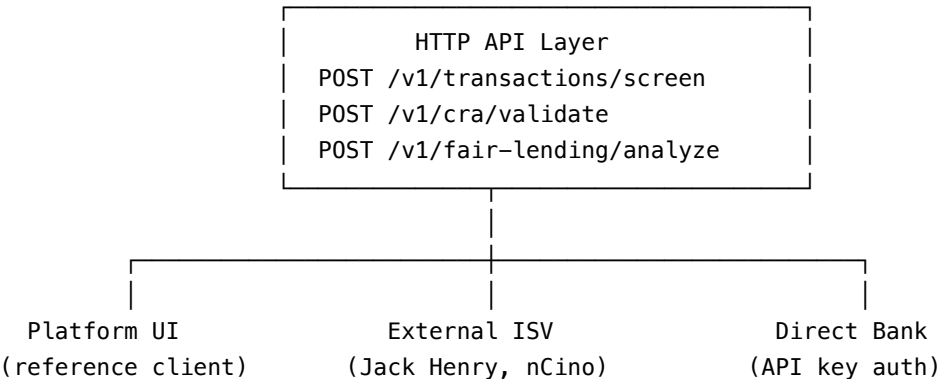
Econofi Compliance Agents: HTTP API Layer Specification

Module: Cross-Cutting API Layer **Version:** 1.0 **Applies To:** All compliance modules — BSA/AML, CRA DataGuard, CRA NarrativeWriter, LIHTC/NMTC ComplianceMonitor, Fair Lending **Strategic Basis:** API-first architecture enables the platform UI and external ISV/bank consumers to use identical endpoints, with zero logic duplication.

Last updated: March 12, 2026

Executive Summary

Every compliance agent in this platform is exposed through a clean HTTP API layer. The platform UI never calls agent logic directly — it always goes through the HTTP API. This design principle, enforced from day one of implementation, means the external API product requires no refactoring: it is the same API, governed by the same auth, rate limiting, and audit trail.



Why API-First Matters Here

Compliance buyers (Chief Compliance Officers, BSA Officers) are not API consumers — they need a platform UI. But the banks they work at have growing developer teams, and ISVs serving those banks need compliance modules they can embed. Building the API as an afterthought to the platform requires expensive refactoring and creates two code paths. Building the platform UI as a thin client on top of the API means both products are production-proven on day one.

Dual Consumer Design

Consumer	Auth Method	Pricing Model
Platform UI (internal)	Service token (env var)	Bundled in platform subscription
Bank developer	JWT with bank_id claim	Platform subscription includes API access
ISV / OEM partner	API key + HMAC signing	Usage-based or OEM license
Sandbox (testing)	Sandbox API key	Free tier, no real data

Design Principles

1. Same API, Dual Consumers

The platform UI is the reference client for the API. Every endpoint tested by the platform is available to external consumers. No internal shortcuts or direct service-layer calls from the UI layer.

2. Zero PII at the API Boundary

All PII sanitization happens in the Orchestrator layer before reaching any API endpoint. The API receives and returns tokenized data only. This is the same PII-zero boundary defined in the agent specs — the API layer enforces it at the transport level.

3. Bank-Scoped by Default

Every request is scoped to a single bank tenant. The JWT `bank_id` claim is extracted by middleware and set as `app.current_bank_id` in PostgreSQL, activating all Row Level Security policies automatically. There is no way for one bank to access another bank's data through the API.

4. Async Where Needed, Sync Where Possible

Single-record operations (screen one transaction, validate one loan record) are synchronous — respond in under 200ms. Batch operations and full portfolio analyses are async — return a `job_id` immediately, deliver results via webhook or polling endpoint.

5. Idempotent POST Operations

All POST endpoints that create or persist data accept an `X-Idempotency-Key` header. Re-submitting the same key within 24 hours returns the original response without re-running the analysis. Prevents duplicate SAR submissions and double billing on retries.

6. Versioned from Day One

All endpoints are prefixed `/v1/`. When breaking changes are needed, `/v2/` is introduced. `/v1/` remains active for a minimum 12-month deprecation window with email notice to all registered API consumers.

Authentication and Authorization

JWT Structure

All authenticated API calls carry a short-lived JWT in the `Authorization: Bearer <token>` header.

```
interface EconofiJWTpayload {
  sub: string;           // User ID (platform user) or service account ID
  bank_id: string;       // Tenant identifier – drives RLS scoping
  scope: ApiScope[];     // Permissions granted
  iat: number;           // Issued at (Unix timestamp)
  exp: number;           // Expiration (Unix timestamp, max 1 hour)
  jti: string;           // JWT ID – used for idempotency and audit logging
}

type ApiScope =
  | 'transactions:read'
  | 'transactions:write'
  | 'transactions:batch'
  | 'alerts:read'
  | 'alerts:write'
  | 'cra:read'
  | 'cra:write'
  | 'cra:batch'
  | 'narrative:read'      // CRA NarrativeWriter – read generated narratives
  | 'narrative:write'     // CRA NarrativeWriter – generate narratives and acknowledge drafts
  | 'lihtc-nmtc:read'     // ComplianceMonitor – read portfolio and alerts
  | 'lihtc-nmtc:write'    // ComplianceMonitor – run scans and create certifications
  | 'fair-lending:read'
  | 'fair-lending:write'
  | 'reports:read'
  | 'admin';
```

API Key Authentication (ISV/OEM)

ISV partners use long-lived API keys with HMAC-SHA256 request signing. The API key identifies the bank tenant; no JWT required.

X-API-Key: ek_live_abc123...

X-Timestamp: 1741651200

X-Signature: HMAC-SHA256(secret, "POST\n/v1/transactions/screen\n1741651200\n{body}")

Middleware Stack (Express/Fastify)

```
// Every request passes through this stack in order
const apiMiddlewareStack = [
  rateLimiter,           // 1. Reject if over rate limit
  authenticateRequest,   // 2. Validate JWT or API key signature
```

```

extractBankId,      // 3. Set bank_id from token claims
setRLSContext,      // 4. SET app.current_bank_id in PostgreSQL connection
auditLogRequest,    // 5. Write to audit_log table (every request, pre-handler)
validateIdempotency, // 6. Check X-Idempotency-Key, return cached response if seen
validateRequestBody, // 7. JSON schema validation
// ... route handler
auditLogResponse,   // 8. Append response code to audit_log row
];

```

Standard Response Envelope

All API responses use this envelope regardless of success or failure:

```

// Present only on NarrativeWriter (POST /v1/cra/narrative) and
// ComplianceMonitor (POST /v1/lihtc-nmtc/scan, POST /v1/lihtc-nmtc/investments/:id/certifications) resp
// All draft outputs require human acknowledgment before regulatory submission.
interface DraftStatus {
    is_draft: boolean;           // true until acknowledged
    human_review_required: boolean; // true until acknowledged
    review_notice: string;       // 'DRAFT – Human Review Required Before Submission'
    acknowledged_at?: string;     // ISO 8601 – set when POST /:id/acknowledge is called
    acknowledged_by?: string;     // User ID of reviewing officer
}

interface ApiResponse<T> {
    success: boolean;
    data: T | null;
    error: ApiError | null;
    draft_status?: DraftStatus;    // Present on all NarrativeWriter and ComplianceMonitor responses
    meta: {
        request_id: string;        // UUID, matches audit log
        bank_id: string;           // Confirming tenant scope
        api_version: string;       // 'v1'
        timestamp: string;         // ISO 8601
        processing_ms?: number;    // Latency of handler (not transport)
        cached_response?: boolean; // true when idempotency key returned a cached result
    };
}

interface ApiError {
    code: ApiErrorCode;
    message: string;              // Human-readable, safe to surface to end users
    details?: Record<string, unknown>; // Field-level validation errors, etc.
    regulatory_reference?: string;  // When error relates to a specific regulation
}

```

```

type ApiErrorCode =
  | 'INVALID_REQUEST'           // 400 - request body fails schema validation
  | 'AUTHENTICATION_REQUIRED'   // 401 - missing or expired token
  | 'FORBIDDEN'                 // 403 - valid token, insufficient scope
  | 'NOT_FOUND'                 // 404 - resource not found in this bank's tenant
  | 'CONFLICT'                  // 409 - idempotency key collision with different body
  | 'RATE_LIMIT_EXCEEDED'       // 429
  | 'AGENT_PROCESSING_ERROR'     // 500 - Claude agent returned an unexpected result
  | 'DATABASE_ERROR'            // 500 - PostgreSQL error
  | 'PII_DETECTED'              // 422 - request body contains suspected PII (rejected at boundary)
  | 'COMPLIANCE_GATE_BLOCKED'    // 451 - legal review gate blocked the operation
  | 'UPSTREAM_CRITICAL_ERRORS'   // 422 - NarrativeWriter called with source CRA job that has critical_
  | 'PIPELINE_DEPENDENCY_UNMET'; // 422 - required upstream module output is missing or incomplete (gen

```

Rate Limits

Consumer Tier	Requests/minute	Batch jobs/hour	Burst
Sandbox	60	2	100/min for 30 seconds
Starter	300	10	600/min for 30 seconds
Professional	1,000	30	2,000/min for 30 seconds
Enterprise	10,000	unlimited	20,000/min for 30 seconds

Rate limit headers on every response:

```

X-RateLimit-Limit: 300
X-RateLimit-Remaining: 247
X-RateLimit-Reset: 1741651260

```

BSA/AML Endpoints

POST /v1/transactions/screen

Screen a single transaction for suspicious activity patterns. Synchronous, SLA <200ms.

```

// Request
interface ScreenTransactionRequest {
  transaction: SanitizedTransaction; // See BSA/AML TransactionMonitor spec for type
  customer_context?: CustomerHistoricalContext;
  options?: {
    detection_sensitivity?: 'standard' | 'elevated' | 'conservative'; // default: 'standard'
    include_explanation?: boolean; // default: true - include human-readable reasoning
  };
}

```

```
// Response data field
interface ScreenTransactionResponse {
  alert: SuspiciousActivityAlert | null; // null = no suspicious activity detected
  screening_id: string; // Unique ID for this screening event
  checked_patterns: string[]; // Which detection patterns were evaluated
  processing_ms: number;
}
```

Status codes: 200 (screened, alert or null), 422 (PII detected in request body), 400 (invalid request), 401/403 (auth).

Example (no alert):

```
{
  "success": true,
  "data": {
    "alert": null,
    "screening_id": "scr_20260311_a1b2c3",
    "checked_patterns": ["structuring", "velocity_anomaly", "round_dollar", "geographic_risk"],
    "processing_ms": 143
  },
  "error": null,
  "meta": { "request_id": "req_xyz", "bank_id": "bank_liberty_001", "api_version": "v1", "timestamp": "2026-03-11T15:00:00Z" }
}
```

POST /v1/transactions/batch

Submit a batch of up to 50,000 transactions for screening. Async — returns `job_id` immediately.

```
// Request
interface BatchScreenRequest {
  transactions: SanitizedTransaction[]; // 1 to 50,000
  customer_contexts?: CustomerHistoricalContext[];
  webhook_url?: string; // POST callback when job completes
  options?: {
    detection_sensitivity?: 'standard' | 'elevated' | 'conservative';
    priority?: 'standard' | 'high'; // high = jump queue, counts toward burst limit
  };
}

// Immediate response (202 Accepted)
interface BatchJobCreatedResponse {
  job_id: string; // Format: job_bsa_YYYY-MM-DD-NNNNN
  status: 'queued';
  transaction_count: number;
  estimated_completion_seconds: number;
  poll_url: string; // GET /v1/transactions/batch/:job_id
}
```

```
}
```

Status codes: 202 (job accepted), 400 (too many transactions, invalid format), 413 (payload too large).

GET /v1/transactions/batch/:job_id

Poll for batch job status and results.

```
interface BatchJobStatusResponse {
  job_id: string;
  status: 'queued' | 'processing' | 'completed' | 'failed';
  transaction_count: number;
  processed_count: number;
  alert_count: number;
  alerts: SuspiciousActivityAlert[]; // populated when status = 'completed'
  error?: string;                  // populated when status = 'failed'
  started_at?: string;
  completed_at?: string;
}
```

GET /v1/alerts

List suspicious activity alerts for the authenticated bank. Supports filtering and pagination.

```
// Query parameters
interface ListAlertsParams {
  severity?: 'low' | 'medium' | 'high' | 'critical';
  status?: 'pending' | 'in_progress' | 'sar_filed' | 'no_sar_warranted' | 'false_positive';
  from_date?: string; // ISO date
  to_date?: string;   // ISO date
  page?: number;      // default: 1
  per_page?: number;  // default: 25, max: 100
}

interface ListAlertsResponse {
  alerts: SuspiciousActivityAlert[];
  pagination: {
    total: number;
    page: number;
    per_page: number;
    total_pages: number;
  };
}
```

PATCH /v1/alerts/:alert_id

Update alert investigation status. Used by compliance officers through the platform UI or directly by API consumers.

```
interface UpdateAlertRequest {
  status: 'in_progress' | 'sar_filed' | 'no_sar_warranted' | 'false_positive';
  investigation_notes?: string;
  sar_reference_number?: string; // Required when status = 'sar_filed'
}
```

Audit: Every status change is written to `alert_audit_log` with officer token, timestamp, and previous state. This record is immutable.

CRA Endpoints

POST /v1/cra/validate

Validate a single CRA loan record against 12 CFR §228.42 requirements. Synchronous.

```
// Request
interface ValidateCRARecordRequest {
  record: SanitizedLoanRecord; // See CRA DataGuard spec for type
  options?: {
    auto_correct?: boolean; // default: true - apply high-confidence corrections
    correction_threshold?: number; // default: 0.80 - minimum confidence to auto-correct
  };
}

// Response data field
interface ValidateCRARecordResponse {
  validation_id: string;
  is_valid: boolean;
  exceptions: CRAValidationException[]; // Empty array = no issues found
  corrections_applied: CRAAutoCorrection[];
  regulatory_citations: string[]; // e.g., ['12 CFR §228.42(b)(1)']
  processing_ms: number;
}

interface CRAValidationException {
  field: string;
  severity: 'CRITICAL' | 'HIGH' | 'MEDIUM' | 'LOW';
  code: string; // e.g., 'CENSUS_TRACT_INVALID'
  message: string; // Human-readable description
  current_value: unknown;
  expected_format?: string;
  regulatory_citation: string;
  blocks_submission: boolean; // true when severity = 'CRITICAL'
}
```



```

}

interface CRAAutoCorrection {
  field: string;
  original_value: unknown;
  corrected_value: unknown;
  confidence: number; // 0-1
  correction_type: string; // e.g., 'CENSUS_TRACT_FORMAT', 'STATE_CODE_NORMALIZATION'
  regulatory_basis: string;
}

```

POST /v1/cra/batch

Submit a batch of up to 10,000 CRA loan records. Async, SLA <5 seconds per 10,000 records.

```

interface BatchValidateCRARequest {
  records: SanitizedLoanRecord[]; // 1 to 10,000
  submission_period: string; // YYYY-MM format - CRA submission period
  webhook_url?: string;
  options?: {
    auto_correct?: boolean;
    correction_threshold?: number;
    stop_on_critical?: boolean; // default: false - continue processing after CRITICAL exceptions
  };
}

// Immediate response (202 Accepted)
interface CRABatchJobCreatedResponse {
  job_id: string; // Format: job_cra_YYYY-MM-DD-NNNNN
  status: 'queued';
  record_count: number;
  estimated_completion_seconds: number;
  poll_url: string;
}

```

GET /v1/cra/reports/:job_id

Retrieve a completed CRA validation report.

```

interface CRAValidationReport {
  job_id: string;
  submission_period: string;
  status: 'queued' | 'processing' | 'completed' | 'failed';
  record_count: number;
}

```

```

valid_count: number;
exception_count: number;
critical_exception_count: number;
corrections_applied_count: number;
ready_for_submission: boolean; // true only when critical_exception_count = 0
exceptions_by_severity: Record<'CRITICAL' | 'HIGH' | 'MEDIUM' | 'LOW', number>;
exceptions_by_field: Record<string, number>;
records_with_exceptions: Array<{
  record_id: string;
  exceptions: CRAValidationException[];
  corrections_applied: CRAAutoCorrection[];
}>;
audit_trail_id: string; // References immutable audit_log table
generated_at?: string;
}

```

CRA Narrative Endpoints

POST /v1/cra/narrative

Generate a CRA performance narrative from a validated DataGuard report. Async — invokes NarrativeWriter agent (claude-opus-4-6, temperature 0.3). **Requires the source DataGuard job to have ready_for_submission: true (zero critical exceptions).** Calling this endpoint with a source job that has unresolved critical exceptions returns 422 UPSTREAM_CRITICAL_ERRORS.

```

// Request
interface GenerateCRANarrativeRequest {
  source_job_id: string; // job_id from a completed /v1/cra/batch run with ready_for_submission: true
  assessment_areas: AssessmentAreaInput[];
  community_development_services?: CommunityDevelopmentServiceInput[];
  community_development_investments?: CommunityDevelopmentInvestmentInput[];
  submission_period: string; // YYYY-MM format
  webhook_url?: string;
}

interface AssessmentAreaInput {
  geoid: string; // FFIEC census delineation ID
  name: string; // e.g., 'Chicago-Naperville-Elgin, IL-IN-WI MSA'
  state_code: string;
}

// Immediate response (202 Accepted)
interface NarrativeJobCreatedResponse {
  job_id: string; // Format: job_cra_nar_YYYY-MM-DD-NNNNN
  status: 'queued';
  source_job_id: string;
}

```

```

    estimated_completion_seconds: number;
    poll_url: string;
    draft_notice: string; // 'All generated narratives are DRAFT and require human review before submission'
}

```

Status codes: 202 (queued), 422 UPSTREAM_CRITICAL_ERRORS (source job has critical_exception_count > 0 or ready_for_submission: false), 400 (invalid request), 401/403 (auth).

Scope required: cra:write, narrative:write

GET /v1/cra/narrative/:job_id

Retrieve generated CRA narrative sections and assembled CRA public file. All responses include draft_status in the envelope until acknowledged.

```

interface CRANarrativeReport {
    job_id: string;
    source_job_id: string;
    submission_period: string;
    status: 'queued' | 'processing' | 'completed' | 'failed';

    // Populated when status = 'completed'
    narrative_sections?: CRANarrativeSection[];
    public_file?: CRAPublicFileOutput;
    examiner_qa?: ExaminerQA[];

    generated_at?: string;
}

interface CRANarrativeSection {
    section_id: string;
    test_type: 'lending_test' | 'investment_test' | 'service_test' | 'community_development_test';
    assessment_area_geoid: string;
    title: string;
    content: string; // Narrative prose – DRAFT until acknowledged
    data_sources: string[];
    regulatory_basis: string[]; // e.g., ['12 CFR §228.21', '12 CFR §228.22']
}

interface CRAPublicFileOutput {
    file_id: string;
    sections: string[]; // Section titles assembled in §228.43 order
    generated_at: string;
}

```

Envelope: Response always includes draft_status field. is_draft: true until POST /acknowledge is called.

Scope required: cra:read, narrative:read

POST /v1/cra/narrative/:job_id/acknowledge

Human review acknowledgment — records that a compliance officer has reviewed the DRAFT narrative before CRA public file submission. Does NOT constitute regulatory approval; regulatory submission remains the bank's responsibility.

```
interface NarrativeAcknowledgeRequest {
  reviewer_notes?: string;
  sections_reviewed: string[]; // section_id list – confirms each section was reviewed
}

interface NarrativeAcknowledgeResponse {
  job_id: string;
  acknowledged_at: string; // ISO 8601
  acknowledged_by: string; // User ID from JWT
}
```

Status codes: 200, 404 (job not found), 409 (already acknowledged), 401/403 (auth).

Scope required: narrative:write, admin

Audit: Written to immutable narrative_acknowledgment_log — no UPDATE or DELETE permitted via RLS.

LIHTC/NMTC Endpoints

POST /v1/lihtc-nmtc/scan

Scan the tax credit investment portfolio for covenant compliance. Async — invokes Compliance-Monitor agent. Generates deadline alerts at configurable thresholds (default: 90/60/30/7 days and overdue) and drafts certification documents.

```
// Request
interface LIHTCNMTCScanRequest {
  investments?: LIHTCNMTCInvestmentInput[]; // If omitted, scans all active investments for the bank
  as_of_date?: string; // ISO date – default: today. Useful for historical audit runs.
  webhook_url?: string;
  options?: {
    alert_thresholds?: number[]; // Days remaining to trigger alerts – default: [90, 60, 30]
    include_draft_certifications?: boolean; // default: true
  };
}

interface LIHTCNMTCInvestmentInput {
  investment_id: string; // Bank's internal investment identifier
}
```

```

    credit_type: 'LIHTC' | 'NMTc' | 'BOTH';
    project_name: string;
    total_investment_amount: number;    // USD
    credit_amount: number;              // USD – tax credits at risk for recapture calculation
    placed_in_service_date: string;     // ISO date – LIHTC: IRC §42 15-year compliance period starts here
    nmtc_closing_date?: string;         // ISO date – NMTc: IRC §45D 7-year period starts here
    covenants: CovenantInput[];
}

interface CovenantInput {
    covenant_type: string;    // See ComplianceMonitor spec for full enumeration
    due_date: string;        // ISO date
    description?: string;
}

// Immediate response (202 Accepted)
interface LIHTCNMTcScanJobCreatedResponse {
    job_id: string;    // Format: job_tc_YYYY-MM-DD-NNNNN
    status: 'queued';
    investment_count: number;
    estimated_completion_seconds: number;
    poll_url: string;
}

```

Status codes: 202 (queued), 400 (invalid request), 401/403 (auth).

Scope required: lihtc-nmtc:write

Envelope: Response includes `draft_status` when `include_draft_certifications: true` (default).

GET /v1/lihtc-nmtc/portfolio

Retrieve the compliance summary for all active tax credit investments. Sorted by urgency — overdue first, then ascending by `days_remaining`.

```

interface LIHTCNMTcPortfolioResponse {
    bank_id: string;
    as_of_date: string;
    total_investments: number;
    total_credits_at_risk: number;    // USD – sum across all active investments

    compliance_summary: {
        compliant: number;
        alerts_active: number;
        overdue: number;
        certifications_pending_review: number;
    };
};

```

```

alerts: ComplianceAlert[];
last_scan_at?: string;      // ISO 8601 – when the most recent scan completed
last_scan_job_id?: string;
}

interface ComplianceAlert {
  alert_id: string;
  investment_id: string;
  project_name: string;
  credit_type: 'LIHTC' | 'NMTc';
  covenant_type: string;
  due_date: string;
  days_remaining: number;    // Negative = overdue
  severity: 'info' | 'warning' | 'critical' | 'overdue';
  estimated_credits_at_risk: number; // USD
  alert_message: string;
  regulatory_reference: string;    // e.g., 'IRC §42(i)(1)', 'IRC §45D(b)(1)(B)'
}

```

Envelope: Includes `draft_status` when any certification in the portfolio is pending review.

Scope required: `lihtc-nmtc:read`

POST /v1/lihtc-nmtc/investments/:investment_id/certifications

Create a DRAFT certification document for a specific investment covenant. ComplianceMonitor generates the draft — human review and acknowledgment required before submission to any regulatory body.

```

interface CreateCertificationRequest {
  covenant_type: string;    // Covenant type requiring certification
  certification_period: string; // YYYY or YYYY-MM for the period being certified
  webhook_url?: string;
}

// Response (202 Accepted – async generation)
interface CertificationJobCreatedResponse {
  certification_id: string;
  investment_id: string;
  status: 'generating';
  draft_notice: string;    // 'DRAFT – Human review and acknowledgment required before submission.'
  estimated_completion_seconds: number;
}

```

Scope required: `lihtc-nmtc:write`

POST /v1/lihtc-nmtc/certifications/:certification_id/acknowledge

Human review acknowledgment for a DRAFT certification. Records that a compliance officer reviewed the document. Does NOT constitute regulatory approval.

```
interface CertificationAcknowledgeRequest {
  reviewer_notes?: string;
}

interface CertificationAcknowledgeResponse {
  certification_id: string;
  investment_id: string;
  acknowledged_at: string; // ISO 8601
  acknowledged_by: string; // User ID from JWT
}
```

Status codes: 200, 404 (certification not found), 409 (already acknowledged), 401/403 (auth).

Scope required: lihtc-nmtc:write, admin

Audit: Written to immutable certification_acknowledgment_log — no UPDATE or DELETE permitted via RLS.

Fair Lending Endpoints

POST /v1/fair-lending/analyze

Analyze a loan portfolio for disparate impact and fair lending violations. Async — analysis involves regression modeling and statistical testing.

```
// Request
interface FairLendingAnalysisRequest {
  loans: SanitizedLoanApplication[]; // See Fair Lending LoanDataAnalyzer spec for type
  analysis_period: {
    from_date: string; // ISO date
    to_date: string; // ISO date
  };
  protected_classes: ProtectedClass[]; // Which bases to analyze
  webhook_url?: string;
  options?: {
    confidence_level?: 0.90 | 0.95 | 0.99; // default: 0.95 – statistical significance threshold
    include_matched_pairs?: boolean; // default: true
    include_examiner_qa?: boolean; // default: true – generate STAR-format Q&A
    legal_review_required?: boolean; // default: true – triggers LegalReviewGate on findings
  };
}

type ProtectedClass =
  | 'race'
```

```

| 'color'
| 'national_origin'
| 'religion'
| 'sex'
| 'familial_status'
| 'disability'
| 'age';

// Immediate response (202 Accepted)
interface FairLendingJobCreatedResponse {
  job_id: string; // Format: job_fl_YYYY-MM-DD-NNNNN
  status: 'queued';
  loan_count: number;
  estimated_completion_seconds: number;
  poll_url: string;
  legal_review_required: boolean; // Informational – if true, results may be blocked pending counsel
}

```

GET /v1/fair-lending/reports/:job_id

Retrieve a completed fair lending analysis report.

```

interface FairLendingAnalysisReport {
  job_id: string;
  status: 'queued' | 'processing' | 'completed' | 'failed' | 'legal_review_pending';
  analysis_period: { from_date: string; to_date: string };
  loan_count: number;

  // Populated when status = 'completed' or 'legal_review_pending'
  disparate_impact_results?: DisparateImpactResult[];
  regression_results?: RegressionAnalysisResult[];
  matched_pair_findings?: MatchedPairFinding[];
  examiner_qa?: ExaminerQA[];

  // Legal gate status
  legal_review_status?: 'not_required' | 'pending' | 'approved' | 'blocked';
  legal_review_notes?: string;

  overall_risk_level?: 'low' | 'medium' | 'high' | 'critical';
  regulatory_citations: string[]; // e.g., ['15 USC §1691', '42 USC §3605']
  generated_at?: string;
}

interface DisparateImpactResult {
  protected_class: ProtectedClass;
  protected_class_value: string; // e.g., 'Hispanic' for national_origin
}

```



```

comparison_group: string;
protected_class_approval_rate: number; // 0-1
comparison_group_approval_rate: number; // 0-1
disparate_impact_ratio: number;          // protected / comparison
passes_80_percent_rule: boolean;         // ratio >= 0.80
chi_square_statistic: number;
p_value: number;
statistically_significant: boolean;      // p_value < (1 - confidence_level)
sample_size: { protected_class: number; comparison_group: number };
regulatory_citation: string;             // '15 USC §1691, 42 USC §3605'
}

```

GET /v1/fair-lending/examiner-prep/:job_id

Retrieve examiner-ready Q&A in STAR format. Requires completed analysis and legal review approval if legal gate was triggered.

```

interface ExaminerPrepResponse {
  job_id: string;
  examiner_qa: ExaminerQA[];
  methodology_summary: string;
  data_sources: string[];
  limitations: string[];
  prepared_at: string;
}

interface ExaminerQA {
  question: string;
  answer: {
    situation: string;
    task: string;
    action: string;
    result: string;
  };
  supporting_data: Record<string, unknown>;
  regulatory_references: string[];
}

```

Async Job Webhook Payload

When a `webhook_url` is provided, the platform POSTs this payload to the URL when the job completes:

```

interface WebhookPayload {
  event: 'job.completed' | 'job.failed' | 'legal_review.required' | 'draft.ready_for_review';
  job_id: string;
  module: 'bsa-aml' | 'cra-validation' | 'cra-narrative' | 'lihtc-nmtc' | 'fair-lending';
  bank_id: string;
  timestamp: string;
  poll_url: string; // Where to retrieve the full results
  summary: {
    status: string;
    record_count: number;
    alert_count?: number; // BSA/AML only
    exception_count?: number; // CRA validation only
    narrative_sections_count?: number; // CRA narrative only
    investment_count?: number; // LIHTC/NMTC only
    findings_count?: number; // Fair Lending only
    draft_review_required?: boolean; // true for NarrativeWriter and ComplianceMonitor outputs
  };
}

```

Webhooks are signed with HMAC-SHA256 using the consumer's signing secret:

X-Econofi-Signature: sha256=<hex>

X-Econofi-Timestamp: 1741651200

Consumers must verify the signature and reject requests where the timestamp is more than 300 seconds old.

Audit Trail

Every API request and response is written to the `api_audit_log` table. This log is append-only and immutable — no UPDATE or DELETE is permitted via RLS.

```

CREATE TABLE public.api_audit_log (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  bank_id     TEXT          NOT NULL,
  request_id  UUID          NOT NULL UNIQUE,
  endpoint    TEXT          NOT NULL, -- e.g., 'POST /v1/transactions/screen'
  http_method TEXT          NOT NULL,
  api_version TEXT          NOT NULL DEFAULT 'v1',
  caller_token TEXT         NOT NULL, -- Anonymized identifier for the calling user/service
  scope_used  TEXT[],
  idempotency_key TEXT,
  request_body_hash TEXT    NOT NULL, -- SHA-256 of request body – not the body itself
  response_code INTEGER    NOT NULL,
  processing_ms INTEGER,
  error_code  TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-- Immutable via RLS: no UPDATE, no DELETE
ALTER TABLE public.api_audit_log ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Banks can view their own audit log"
  ON public.api_audit_log
  FOR SELECT
  USING (bank_id = current_setting('app.current_bank_id'));

-- No INSERT policy via app – written by service account only
-- No UPDATE or DELETE policy – intentionally absent

```

Data retention: 5 years (matches BSA/AML requirement, longest of the three modules).

Test Specifications

```

// specs/api/api-layer.test.ts

describe('Authentication middleware', () => {
  it('rejects requests with no Authorization header with 401', async () => {
    const res = await request(app).post('/v1/transactions/screen').send(validPayload);
    expect(res.status).toBe(401);
    expect(res.body.error.code).toBe('AUTHENTICATION_REQUIRED');
  });

  it('rejects expired JWTs with 401', async () => {
    const expiredToken = generateJWT({ exp: Math.floor(Date.now() / 1000) - 60 });
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${expiredToken}`)
      .send(validPayload);
    expect(res.status).toBe(401);
  });

  it('rejects tokens with wrong bank_id scope for the resource with 403', async () => {
    const token = generateJWT({ bank_id: 'bank_other_999', scope: ['transactions:write'] });
    // bank_other_999 cannot access bank_liberty_001 data via RLS
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${token}`)
      .send({ ...validPayload, bank_id: 'bank_liberty_001' });
    expect(res.status).toBe(403);
  });

  it('rejects tokens missing required scope with 403', async () => {
    const token = generateJWT({ bank_id: 'bank_test', scope: ['alerts:read'] }); // missing transactions

```

```

    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${token}`)
      .send(validPayload);
    expect(res.status).toBe(403);
    expect(res.body.error.code).toBe('FORBIDDEN');
  });

  it('accepts valid HMAC-signed API key request', async () => {
    const headers = generateApiKeyHeaders(validApiKey, validApiSecret, 'POST', '/v1/transactions/screen')
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set(headers)
      .send(validPayload);
    expect(res.status).toBe(200);
  });

  it('rejects API key requests with timestamp older than 300 seconds', async () => {
    const staleHeaders = generateApiKeyHeaders(validApiKey, validApiSecret, 'POST', '/v1/transactions/screen')
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set(staleHeaders)
      .send(validPayload);
    expect(res.status).toBe(401);
  });
});

describe('PII boundary enforcement', () => {
  it('rejects transaction body containing SSN pattern with 422 PII_DETECTED', async () => {
    const piiPayload = { ...validPayload, transaction: { ...validPayload.transaction, description: 'SSN' } }
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .send(piiPayload);
    expect(res.status).toBe(422);
    expect(res.body.error.code).toBe('PII_DETECTED');
  });

  it('rejects transaction body containing account number pattern with 422 PII_DETECTED', async () => {
    const piiPayload = { ...validPayload, transaction: { ...validPayload.transaction, description: 'Account' } }
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .send(piiPayload);
    expect(res.status).toBe(422);
    expect(res.body.error.code).toBe('PII_DETECTED');
  });
});

```

```

it('accepts a properly sanitized transaction', async () => {
  const res = await request(app)
    .post('/v1/transactions/screen')
    .set('Authorization', `Bearer ${validToken}`)
    .send(validSanitizedPayload);
  expect(res.status).toBe(200);
});
});

describe('Idempotency', () => {
  it('returns the same response for duplicate idempotency key', async () => {
    const key = 'idem_test_001';
    const res1 = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .set('X-Idempotency-Key', key)
      .send(validPayload);
    const res2 = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .set('X-Idempotency-Key', key)
      .send(validPayload);
    expect(res1.body.data.screening_id).toBe(res2.body.data.screening_id);
    expect(res2.body.meta).toMatchObject({ cached_response: true });
  });

  it('returns 409 CONFLICT when same idempotency key sent with different body', async () => {
    const key = 'idem_conflict_001';
    await request(app).post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .set('X-Idempotency-Key', key)
      .send(validPayload);
    const res = await request(app).post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .set('X-Idempotency-Key', key)
      .send({ ...validPayload, transaction: { ...validPayload.transaction, amount: 99999 } });
    expect(res.status).toBe(409);
    expect(res.body.error.code).toBe('CONFLICT');
  });
});

describe('Rate limiting', () => {
  it('returns 429 after exceeding the configured rate limit', async () => {
    const requests = Array.from({ length: 61 }, () =>
      request(app).post('/v1/transactions/screen')
        .set('Authorization', `Bearer ${sandboxToken}`)
        .send(validPayload)
    );
  });
});

```

```

    );
    const responses = await Promise.all(requests);
    const tooManyRequests = responses.filter(r => r.status === 429);
    expect(tooManyRequests.length).toBeGreaterThan(0);
  });

  it('includes X-RateLimit-Remaining header on every response', async () => {
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .send(validPayload);
    expect(res.headers['x-ratelimit-remaining']).toBeDefined();
    expect(parseInt(res.headers['x-ratelimit-remaining'])).toBeGreaterThanOrEqual(0);
  });
});

describe('Response envelope', () => {
  it('every successful response includes meta.request_id, meta.bank_id, meta.api_version', async () => {
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .send(validPayload);
    expect(res.body.meta.request_id).toMatch(/^req_/);
    expect(res.body.meta.bank_id).toBe('bank_test');
    expect(res.body.meta.api_version).toBe('v1');
  });

  it('every error response includes error.code and error.message', async () => {
    const res = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .send({ invalid: 'body' });
    expect(res.body.error.code).toBeDefined();
    expect(res.body.error.message).toBeDefined();
    expect(typeof res.body.error.message).toBe('string');
  });
});

describe('Audit logging', () => {
  it('writes an audit_log row for every API request including failures', async () => {
    const requestId = await request(app)
      .post('/v1/transactions/screen')
      .set('Authorization', `Bearer ${validToken}`)
      .send(validPayload)
      .then(r => r.body.meta.request_id);

    const logRow = await db.query(

```

```

        'SELECT * FROM api_audit_log WHERE request_id = $1',
        [requestId]
    );
    expect(logRow.rows).toHaveLength(1);
    expect(logRow.rows[0].bank_id).toBe('bank_test');
    expect(logRow.rows[0].response_code).toBe(200);
});

it('audit_log rows cannot be updated or deleted', async () => {
    await expect(
        db.query('UPDATE api_audit_log SET response_code = 999 WHERE bank_id = $1', ['bank_test'])
    ).rejects.toThrow();

    await expect(
        db.query('DELETE FROM api_audit_log WHERE bank_id = $1', ['bank_test'])
    ).rejects.toThrow();
});

describe('BSA/AML - POST /v1/transactions/screen', () => {
    it('returns null alert for a normal transaction', async () => {
        const res = await request(app)
            .post('/v1/transactions/screen')
            .set('Authorization', `Bearer ${validToken}`)
            .send({ transaction: normalTransaction });
        expect(res.status).toBe(200);
        expect(res.body.data.alert).toBeNull();
    });

    it('returns a high-severity alert for a structuring pattern', async () => {
        const res = await request(app)
            .post('/v1/transactions/screen')
            .set('Authorization', `Bearer ${validToken}`)
            .send({ transaction: structuringTransaction, customer_context: dormantAccountContext });
        expect(res.status).toBe(200);
        expect(res.body.data.alert.severity).toBe('high');
        expect(res.body.data.alert.alert_type).toBe('structuring');
        expect(res.body.data.alert.regulatory_citation).toContain('31 USC §5324');
    });

    it('processes within the 200ms SLA', async () => {
        const start = Date.now();
        await request(app)
            .post('/v1/transactions/screen')
            .set('Authorization', `Bearer ${validToken}`)
            .send({ transaction: normalTransaction });
        expect(Date.now() - start).toBeLessThan(200);
    });
});

```

```

});
});

describe('CRA - POST /v1/cra/validate', () => {
  it('returns is_valid: true for a compliant loan record', async () => {
    const res = await request(app)
      .post('/v1/cra/validate')
      .set('Authorization', `Bearer ${craToken}`)
      .send({ record: compliantLoanRecord });
    expect(res.status).toBe(200);
    expect(res.body.data.is_valid).toBe(true);
    expect(res.body.data.exceptions).toHaveLength(0);
  });

  it('returns CRITICAL exception for invalid census tract code', async () => {
    const res = await request(app)
      .post('/v1/cra/validate')
      .set('Authorization', `Bearer ${craToken}`)
      .send({ record: invalidCensusTractRecord });
    expect(res.status).toBe(200);
    expect(res.body.data.exceptions[0].severity).toBe('CRITICAL');
    expect(res.body.data.exceptions[0].code).toBe('CENSUS_TRACT_INVALID');
    expect(res.body.data.exceptions[0].regulatory_citation).toContain('12 CFR §228.42');
  });

  it('auto-corrects a census tract formatting error when correction is confident', async () => {
    const res = await request(app)
      .post('/v1/cra/validate')
      .set('Authorization', `Bearer ${craToken}`)
      .send({ record: censusTractFormattingErrorRecord, options: { auto_correct: true } });
    expect(res.body.data.corrections_applied).toHaveLength(1);
    expect(res.body.data.corrections_applied[0].confidence).toBeGreaterThanOrEqual(0.80);
    expect(res.body.data.is_valid).toBe(true);
  });
});

describe('Pipeline halt - UPSTREAM_CRITICAL_ERRORS', () => {
  it('returns 422 UPSTREAM_CRITICAL_ERRORS when source CRA job has critical exceptions', async () => {
    const jobWithCriticals = await createCRABatchJobWithCriticals(); // job where critical_exception_cou
    const res = await request(app)
      .post('/v1/cra/narrative')
      .set('Authorization', `Bearer ${narrativeToken}`)
      .send({ source_job_id: jobWithCriticals.job_id, assessment_areas: [validAssessmentArea], submission
    expect(res.status).toBe(422);
    expect(res.body.error.code).toBe('UPSTREAM_CRITICAL_ERRORS');
    expect(res.body.error.regulatory_reference).toContain('12 CFR §228.42');
  });
});

```



```

it('returns 422 UPSTREAM_CRITICAL_ERRORS when source CRA job has ready_for_submission: false', async () => {
  const incompleteJob = await createIncompleteCRABatchJob();
  const res = await request(app)
    .post('/v1/cra/narrative')
    .set('Authorization', `Bearer ${narrativeToken}`)
    .send({ source_job_id: incompleteJob.job_id, assessment_areas: [validAssessmentArea], submission_period: validSubmissionPeriod });
  expect(res.status).toBe(422);
  expect(res.body.error.code).toBe('UPSTREAM_CRITICAL_ERRORS');
});

it('accepts narrative request when source CRA job has ready_for_submission: true', async () => {
  const cleanJob = await createCleanCRABatchJob(); // critical_exception_count = 0, ready_for_submission = true
  const res = await request(app)
    .post('/v1/cra/narrative')
    .set('Authorization', `Bearer ${narrativeToken}`)
    .send({ source_job_id: cleanJob.job_id, assessment_areas: [validAssessmentArea], submission_period: validSubmissionPeriod });
  expect(res.status).toBe(202);
  expect(res.body.data.job_id).toMatch(/^job_cra_nar_/);
});

describe('CRA Narrative - draft_status contract', () => {
  it('GET /v1/cra/narrative/:job_id response envelope includes draft_status.is_draft: true', async () => {
    const { job_id } = await submitAndWaitForNarrativeCompletion(cleanCRAJob, validAssessmentArea);
    const res = await request(app)
      .get(`/v1/cra/narrative/${job_id}`)
      .set('Authorization', `Bearer ${narrativeToken}`);
    expect(res.status).toBe(200);
    expect(res.body.draft_status.is_draft).toBe(true);
    expect(res.body.draft_status.human_review_required).toBe(true);
    expect(res.body.draft_status.review_notice).toContain('DRAFT');
    expect(res.body.draft_status.acknowledged_at).toBeUndefined();
  });

  it('POST /v1/cra/narrative/:job_id/acknowledge requires sections_reviewed array', async () => {
    const res = await request(app)
      .post(`/v1/cra/narrative/${completedNarrativeJobId}/acknowledge`)
      .set('Authorization', `Bearer ${adminToken}`)
      .send({ sections_reviewed: [] }); // empty array - must have at least one section
    expect(res.status).toBe(400);
    expect(res.body.error.code).toBe('INVALID_REQUEST');
  });

  it('POST /v1/cra/narrative/:job_id/acknowledge records acknowledgment and sets acknowledged_at', async () => {
    const res = await request(app)
      .post(`/v1/cra/narrative/${completedNarrativeJobId}/acknowledge`)
      .set('Authorization', `Bearer ${adminToken}`)

```

```

        .send({ sections_reviewed: [validSectionId], reviewer_notes: 'Reviewed – data verified against loa
    expect(res.status).toBe(200);
    expect(res.body.data.acknowledged_at).toBeDefined();
    expect(res.body.data.acknowledged_by).toBe('user_admin_test');
  });

  it('POST /v1/cra/narrative/:job_id/acknowledge returns 409 if already acknowledged', async () => {
    await acknowledgeNarrativeJob(completedNarrativeJobId);
    const res = await request(app)
      .post(`/v1/cra/narrative/${completedNarrativeJobId}/acknowledge`)
      .set('Authorization', `Bearer ${adminToken}`)
      .send({ sections_reviewed: [validSectionId] });
    expect(res.status).toBe(409);
    expect(res.body.error.code).toBe('CONFLICT');
  });
});

describe('LIHTC/NMTC – POST /v1/lihtc-nmtc/scan', () => {
  it('returns 202 Accepted with job_id for a valid portfolio scan request', async () => {
    const res = await request(app)
      .post('/v1/lihtc-nmtc/scan')
      .set('Authorization', `Bearer ${lihtcToken}`)
      .send({ investments: [validLIHTCInvestment, validNMTCInvestment] });
    expect(res.status).toBe(202);
    expect(res.body.data.job_id).toMatch(/^job_tc_/);
    expect(res.body.data.investment_count).toBe(2);
  });

  it('response envelope includes draft_status when scan includes certifications', async () => {
    const { job_id } = await submitAndWaitForScanCompletion([investmentWithNearDueDate]);
    const portfolioRes = await request(app)
      .get('/v1/lihtc-nmtc/portfolio')
      .set('Authorization', `Bearer ${lihtcToken}`);
    expect(portfolioRes.body.draft_status.is_draft).toBe(true);
  });
});

describe('LIHTC/NMTC – GET /v1/lihtc-nmtc/portfolio', () => {
  it('alerts are sorted overdue first, then ascending by days_remaining', async () => {
    const res = await request(app)
      .get('/v1/lihtc-nmtc/portfolio')
      .set('Authorization', `Bearer ${lihtcToken}`);
    expect(res.status).toBe(200);
    const alerts = res.body.data.alerts;
    const overdueAlerts = alerts.filter((a: ComplianceAlert) => a.severity === 'overdue');
    expect(alerts.slice(0, overdueAlerts.length).every((a: ComplianceAlert) => a.days_remaining < 0)).to
  });
});

```

```

it('alert includes regulatory_reference matching credit_type', async () => {
  const res = await request(app)
    .get('/v1/lihtc-nmtc/portfolio')
    .set('Authorization', `Bearer ${lihtcToken}`);
  const lihtcAlert = res.body.data.alerts.find((a: ComplianceAlert) => a.credit_type === 'LIHTC');
  if (lihtcAlert) {
    expect(lihtcAlert.regulatory_reference).toContain('IRC §42');
  }
  const nmtcAlert = res.body.data.alerts.find((a: ComplianceAlert) => a.credit_type === 'NMTC');
  if (nmtcAlert) {
    expect(nmtcAlert.regulatory_reference).toContain('IRC §45D');
  }
});

describe('Fair Lending – POST /v1/fair-lending/analyze', () => {
  it('returns 202 Accepted with job_id for valid analysis request', async () => {
    const res = await request(app)
      .post('/v1/fair-lending/analyze')
      .set('Authorization', `Bearer ${flToken}`)
      .send({ loans: largeLoanPortfolio, analysis_period: { from_date: '2025-01-01', to_date: '2025-12-31' } });
    expect(res.status).toBe(202);
    expect(res.body.data.job_id).toMatch(/^job_fl_/);
    expect(res.body.data.status).toBe('queued');
  });

  it('report includes disparate_impact_ratio and passes_80_percent_rule for each protected class', async () => {
    const { job_id } = await submitAndWaitForCompletion(largeLoanPortfolio);
    const res = await request(app)
      .get(`/v1/fair-lending/reports/${job_id}`)
      .set('Authorization', `Bearer ${flToken}`);
    expect(res.body.data.disparate_impact_results).toBeDefined();
    res.body.data.disparate_impact_results.forEach((result: DisparateImpactResult) => {
      expect(result.disparate_impact_ratio).toBeGreaterThanOrEqual(0);
      expect(typeof result.passes_80_percent_rule).toBe('boolean');
      expect(result.p_value).toBeGreaterThanOrEqual(0);
      expect(result.regulatory_citation).toContain('15 USC §1691');
    });
  });

  it('blocks report access when legal_review_required and review is pending', async () => {
    const { job_id } = await submitWithLegalGate(portfolioWithDisparateImpact);
    const res = await request(app)
      .get(`/v1/fair-lending/examiner-prep/${job_id}`)
      .set('Authorization', `Bearer ${flToken}`);
    expect(res.status).toBe(451);
    expect(res.body.error.code).toBe('COMPLIANCE_GATE_BLOCKED');
  });
});

```

```
});  
});
```

Security Considerations

PII Detection at API Boundary

A lightweight regex scan runs on every POST request body before the handler executes. Patterns checked: SSN (`\d{3}-\d{2}-\d{4}`), account numbers (9+ consecutive digits in suspicious context), full names in free-text fields. Any detection returns 422 `PII_DETECTED` and the request is rejected without processing. The rejection is logged to `api_audit_log` for compliance review.

This is a defense-in-depth measure. The primary PII sanitization is the Orchestrator layer. The API boundary check is a backstop.

Transport Security

TLS 1.3 minimum. HSTS enforced. Certificate pinning for ISV partners on request. No HTTP redirects — reject non-TLS connections at the load balancer.

Data at Rest

Request body hashes (not bodies) are stored in `api_audit_log`. No full request/response bodies are logged to prevent PII leakage through the logging system. Full request bodies exist only in memory during processing.

Versioning and Deprecation Policy

- `/v1/` is the current stable version.
 - When a breaking change is required, `/v2/` is introduced alongside `/v1/`.
 - `/v1/` receives a minimum **12-month deprecation notice** via email to all registered API consumers and a `Deprecation` response header: `Deprecation: Sat, 11 Mar 2027 00:00:00 GMT`.
 - Non-breaking additions (new optional request fields, new response fields) are made in-place within the existing version.
 - Additive changes are announced in the changelog but do not require consumer action.
-

Sprint Day 2 MVP Endpoint Tier

The following endpoints must be functional for the Day 3 MDI compliance officer demo. All other endpoints are post-sprint (Weeks 3–6 of the 8-week roadmap).

Priority	Endpoint	Module	When Required
Sprint Required	POST /v1/cra/validate	CRA DataGuard	Day 1 pipeline
Sprint Required	POST /v1/cra/narrative	CRA NarrativeWriter	Day 1 pipeline output
Sprint Required	POST /v1/lihtc-nmtc/scan	LIHTC/NMTC ComplianceMonitor	Day 2 AM output
Sprint Required	GET /v1/lihtc-nmtc/portfolio	LIHTC/NMTC ComplianceMonitor	Day 2 AM output
Sprint Required	GET /v1/alerts	BSA/AML	Alert visibility for demo
Post-Sprint	POST /v1/transactions/screen	BSA/AML	Weeks 3–4
Post-Sprint	POST /v1/transactions/batch	BSA/AML	Weeks 3–4
Post-Sprint	GET /v1/transactions/batch/:job_id	BSA/AML	Weeks 3–4
Post-Sprint	PATCH /v1/alerts/:alert_id	BSA/AML	Weeks 3–4
Post-Sprint	POST /v1/cra/batch	CRA	Weeks 3–4
Post-Sprint	GET /v1/cra/reports/:job_id	CRA	Weeks 3–4
Post-Sprint	GET /v1/cra/narrative/:job_id	CRA	Weeks 3–4
Post-Sprint	POST /v1/cra/narrative/:job_id/acknowledge	CRA	Weeks 5–6
Post-Sprint	POST /v1/fair-lending/analyze	Fair Lending	Weeks 3–4
Post-Sprint	GET /v1/fair-lending/reports/:job_id	Fair Lending	Weeks 3–4
Post-Sprint	GET /v1/fair-lending/examiner-prep/:job_id	Fair Lending	Weeks 3–4
Post-Sprint	POST /v1/lihtc-nmtc/investments/:id/certifications	LIHTC/NMTC	Weeks 3–4
Post-Sprint	POST /v1/lihtc-nmtc/certifications/:id/acknowledge	LIHTC/NMTC	Weeks 5–6

Day 3 demo validation gate: Officer submits 5 loan records → POST /v1/cra/validate returns clean → POST /v1/cra/narrative generates a narrative section with `draft_status.is_draft: true` → POST /v1/lihtc-nmtc/scan flags a 30-day deadline → GET /v1/alerts shows active alerts. All five Sprint Required endpoints must return valid responses for the demo to succeed.

Implementation Notes

Recommended framework: Fastify (Node.js) — lower overhead than Express, built-in JSON schema validation via Ajv, first-class TypeScript support.

Directory structure for implementation repo:

```
econofi-agents-api/
├── src/
│   ├── middleware/
│   │   ├── authenticate.ts
│   │   ├── extractBankId.ts
│   │   ├── setRLSContext.ts
│   │   ├── auditLog.ts
│   │   ├── idempotency.ts
│   │   ├── rateLimiter.ts
│   │   └── piiDetector.ts
│   ├── routes/
│   │   ├── v1/
│   │   │   ├── transactions.ts
│   │   │   ├── alerts.ts
│   │   │   ├── cra.ts           ← validation + narrative endpoints
│   │   │   ├── lihtcNmtc.ts    ← ComplianceMonitor endpoints
│   │   │   └── fairLending.ts
│   │   └── health.ts
│   ├── services/                ← agent logic lives here, called by routes
│   │   ├── TransactionScreeningService.ts
│   │   ├── CRAValidationService.ts
│   │   ├── CRANarrativeService.ts
│   │   ├── LIHTCNMTCComplianceService.ts
│   │   └── FairLendingAnalysisService.ts
│   └── db/
│       ├── pool.ts
│       └── rlsContext.ts
├── tests/
│   └── api/
│       └── api-layer.test.ts    ← test cases from this spec
└── openapi/
    └── v1.yaml                  ← OpenAPI 3.1 spec (machine-readable contract)
```

The `services/` layer is the boundary between the HTTP routing layer and the Claude agent logic. This is what eventually gets exposed externally — the routes are thin wrappers; the services contain the business logic.

Last updated: March 12, 2026